

GitHub Actions & Frontend Static Deployment

A Clear, Step-by-Step Explanation for Beginners

This reference guide breaks down your production-grade Frontend CI/CD pipeline line-by-line. It compiles static web distribution assets (HTML/JS/CSS), pushes them into an AWS S3 bucket container, and forces a global cache clear across edge servers.

1. Triggers & Concurrency Guardrails

Workflow Name & Triggers

```
name: Frontend deploy

on:
  push:
    branches: [main]
    paths: ["app/frontend/**", ".github/workflows/frontend.yml"]
```

- **Explanation:** This workflow only starts when code changes are pushed or merged into the main branch, and specifically targets modifications inside the `app/frontend/` directory or to this deployment file.

Concurrency Lock Control

```
concurrency:
  group: frontend-${{ github.ref }}
  cancel-in-progress: false
```

- **Explanation:** Updating Content Delivery Networks (CDNs) requires clear timeline separation. If back-to-back updates occur, the second build stands in line cleanly to avoid terminating an ongoing global cache clearance.

2. Step-by-Step Execution Sequence

GitHub sets up a fresh **Ubuntu Linux** cloud runner server and executes the requested blocks sequentially:

Step 1: Code Retrieval & Node Environment Setup

```
- uses: actions/checkout@v4
- uses: actions/setup-node@v4
  with:
    node-version: "20"
    cache: "npm"
```

Action: Clones code parameters and loads **Node.js v20**. Caching mechanics are bound to `package-lock.json` to drastically speed up future execution pipelines.

Step 2: Read Live CDN Target Mappings

```
cd terraform/cdn
terraform init -input=false
{
  echo "BUCKET=$(terraform output -raw frontend_bucket)"
  echo "DIST_ID=$(terraform output -raw cloudfront_distribution_id)"
} >> "$GITHUB_ENV"
```

Action: Temporarily switches context into your Terraform configurations to harvest deployment output strings, isolating your exact AWS S3 bucket name and CloudFront CDN allocation reference.

Step 3: Asset Dependency Packaging & Compilation

```
env:
  VITE_API_URL: ${vars.VITE_API_URL}
run: |
  npm ci
  npm run build
```

Action: Passes your production environment API string token into compilation memory, executes a high-speed dependency tree sync (`npm ci`), and packs optimized static build outputs directly into a `dist/` workspace directory.

Step 4: AWS Storage Synchronization

```
aws S3 sync app/frontend/dist/ "s3://${BUCKET}/" --delete
```

Action: Mirrors your local compiled file arrays directly to your live **AWS S3 hosting bucket**. The critical `--delete` tag identifies old hashed assets remaining from historical builds and purges them safely.

Step 5: CDN Network Invalidation Cache Flush

```
INVALIDATION_ID=$(aws cloudfront create-invalidation --distribution-id "$DIST_ID" --paths "/*" ...)
```

Action: Forces your Content Delivery Network edges to immediately discard any old cached copies of the interface layout. The script monitors validation metrics via loop tracking and logs out success when new site features become live around the globe.

3. Full Architecture Mappings Ecosystem

Workflow Name	Domain Scope	Target Deployment Environment	Trigger Boundary
terraform.yml	Cloud Topography Base Layers	Live Cloud Resource Architecture	Terraform Folder Adjustments
backend.yml	Business Data Application Layer	Docker Containers on Managed Nodes	Backend Code Component Sweeps
frontend.yml	User Client Presentation Interface	AWS S3 File Storage Buckets & CDN Edges	Frontend UI Directory Changes